



RESOURCES

[Download \[Packed\]](#)[Source Repository](#)[Wiki](#)[API](#)[Bug Tracker](#)[Group](#)**Cross Browser, Cross Platform, JavaScript ORM**

ActiveRecord.js makes it easy to begin working with databases in JavaScript on the client or server, with **Google Gears** & **Chrome**, **Aptana Jaxer**, **Adobe AIR** or any platform supporting the **W3C HTML5 SQL Specification** (currently **Webkit** and **iPhone**).

**Contributors**

ActiveRecord.js is the first released component of ActiveJS a JavaScript framework initiative sponsored by **Aptana**. This project is still in it's early stages and we welcome your thoughts and ideas on the future of JavaScript application development. Since this is a GitHub hosted project, if you would like to directly contribute we recommend you fork the project and initiate a pull request to merge your changes.

**Project Features**

ActiveRecord.js is a single file, MIT licensed, relies on no external JavaScript libraries, supports automatic table creation, data validation, data synchronization, relationships between models, life cycle callbacks and can use an in memory hash table to store objects if no SQL database is available.

Setup

To begin using ActiveRecord.js, you will need to include the activerecord.js file and establish a connection, if you do not specify a connection type, one will be automatically chosen.

```
ActiveRecord.connect();
```

You can also specify a specific type of adapter. Jaxer requires pre-configuring of the database for the entire application, and Gears automatically configures the database, so simply passing the type of connection is enough. In all of the SQLite implementations you can optionally specify a database name (browser) or path (Jaxer):

```
ActiveRecord.connect(ActiveRecord.Adapters.InMemory); //JS memory
ActiveRecord.connect(ActiveRecord.Adapters.JaxerMySQL); //Jaxer MySQL
ActiveRecord.connect(ActiveRecord.Adapters.JaxerSQLite); //Jaxer SQLite
ActiveRecord.connect(ActiveRecord.Adapters.AIR); //Adobe AIR
ActiveRecord.connect(ActiveRecord.Adapters.Local, 'my_database'); //Gears or H1
```

Once connected you can always execute SQL statements directly:

```
ActiveRecord.execute('CREATE TABLE IF NOT EXISTS posts (id INTEGER PRIMARY KEY
```

Logging (to either the Jaxer log or browser console) can be turned on by setting:

```
ActiveRecord.logging = true;
```

In Memory Adapter

If you are using a browser or platform that does not have access to a SQL database, you can use the InMemory adapter which will store your objects in memory. All features (including find by SQL) will still work, but you will not be able to use the Migration features, since there are no table schema. Since your objects will not persist, the second parameter to establish a connection is a hash with the data you would like to use in this format: {table_name: {id: row}}. The InMemory adapter will also trigger three observable events that allow you to write an AJAX persistence layer.

```
ActiveRecord.connect(ActiveRecord.Adapters.InMemory,{
  table_one: {
    1: {row_data},
    2: {row_data}
  },
  table_two: {
    1: {row_data},
    2: {row_data}
  }
});

ActiveRecord.connection.observe('created',function(table_name,id,data){});
ActiveRecord.connection.observe('updated',function(table_name,id,data){});
ActiveRecord.connection.observe('destroyed',function(table_name,id){});
```

Defining Your Model

The only rule for all ActiveRecord classes is that the related table in the database must have an auto incrementing 'id' property. If you are working with a database table that already exists, you can create a model pseudo-class using the create() method, passing the table name as the first parameter, and any methods you want to define on that class as the second parameter:

```
var Post = ActiveRecord.create('posts',{
  getWordCount: function(){
    return this.get('text').split(/\s+/).length;
  }
});
```

This both returns the class, and stores it inside ActiveRecord.Models.Post. If the table for your model does not yet exist you can use the define() method which takes the desired table as the first argument, the fields as the second and the methods as the third:

```
var User = ActiveRecord.define('users',{
  username: '',
  password: '',
  post_count: 0,
  profile: {
    type: 'TEXT',
    value: ''
  }
},{
```

```
getFormattedProfile: function(){
    return Markdown.format(this.get('profile'));
}
});
```

Migrations

Migrations are a method of versioning the database schema used by your application. All of your migrations must be defined in an object assigned to ActiveRecord.Migrations.migrations. The keys need not be numerically sequential, but must be numeric (i.e. 1,2,3 or 100,200,300).

Each migration object must have an up() and down() method which will receive an ActiveRecord.Migrations.Schema object. createTable() and addColumn() both use the same syntax as define() to specify default values and field types.

```
ActiveRecord.Migrations.migrations = {
  1: {
    up: function(schema){
      schema.createTable('one',{
        a: '',
        b: {
          type: 'TEXT',
          value: 'default'
        }
      });
    },
    down: function(schema){
      schema.dropTable('one');
    }
  },
  2: {
    up: function(schema){
      schema.addColumn('one','c');
    },
    down: function(schema){
      schema.dropColumn('one','c');
    }
  }
};

ActiveRecord.Migrations.migrate(); //will migrate to the highest available (2)
ActiveRecord.Migrations.migrate(0); //migrates down below 1, effectively erasing
ActiveRecord.Migrations.migrate(1); //migrates to version 1
```

Class & Instance Methods

JavaScript does not have true static methods or classes, but in this case any method of the User variable above is referred to as a class method, and any method of a particular user (that the User class would find) is referred to as an instance method. The most important class methods are create() and find():

```
var jessica = User.create({
  username: 'Jessica',
  password: 'rabbit'
});
```

To add new class or instance methods to all ActiveRecord models in the following way:

```
ActiveRecord.ClassMethods.myClassMethod = function(){/* this == model class */
ActiveRecord.InstanceMethods.myInstanceMethod = function(){/* this == model ir
```

Getters & Setters

It is extremely important to note that all of the attributes/columns of the user are accessible directly for reading (for convenience), but cannot be written directly. You **must** use the set() method to set an attribute, you **should** use the get() method to access all attributes, but you **must** use the get() method if your attribute/column is a method of the object or a JavaScript reserved keyword ('save', 'initialize', 'default', etc).

```
jessica.username // 'Jessica'
jessica.get('username'); // 'Jessica'
jessica.username = 'new username';
jessica.get('username'); // 'Jessica'
jessica.set('username', 'new username');
jessica.get('username'); // 'new username'
```

When Data is Persisted

Data is only persisted to the database in three cases: when you explicitly call save() on a record, when you call create() on a record, or create a child record through a relationship (the method will contain the word "create" in this case), or when you call updateAttribute() on a record. In the case of the latter, only the attribute you update will be saved, the rest of the record will not be persisted to the database, even if changes have been made. Calling save() may add an "id" property to the record if it does not exist, but if there are no errors, it's state will otherwise be unchanged. You can call refresh() on any record to ensure it is not out of synch with your DB at any time.

Finding Records

If you created the User class using the define() method you automatically have free "finder" methods:

```
User.findByUsername('Jessica');
User.findAllByPassword(''); //finds all with blank passwords
```

Otherwise you can use the base find() method, which takes a hash of options, a numeric id or a complete SQL string:

```
var posts = Post.find({
  all: true,
  order: 'id DESC',
  limit: 10
});
```

Synchronization

It is sometimes useful to keep records that have already been found in synch with the database. Each found record has a synchronize() method that will keep the values of that record in synch with the database. If you pass the parameter synchronize: true to find(), all objects will have their values synchronized, and in addition the result set itself will update as objects are destroyed or created. Both features are relatively expensive operations, and are not automatically garbage collected / stopped when the record or result set goes out of scope, so you will need to explicitly stop both record and result set synchronization.

```

var aaron = User.findByName('aaron');
aaron.synchronize();

var aaron_clone = User.findByName('aaron');
aaron_clone.set('name', 'Aaron!');
aaron_clone.save();

aaron.get('name') == 'Aaron!';
aaron.stop(); //record will no longer be synchronized

var users = User.find({
  all: true,
  synchronize: true
});
//users contains aaron
aaron.destroy();
//users will no longer contain aaron
users.stop(); //result set will no longer be synchronized

```

Lifecycle

There are 7 currently supported lifecycle events which allow granular control over your data, and are convenient to build user interface components and interactions around on the client side:

- afterInitialize
- beforeSave
- afterSave
- beforeCreate
- afterCreate
- beforeDestroy
- afterDestroy

beforeSave and afterSave are called when both creating (inserting) and saving (updating) a record. You can observe events on all instances of a class, or just a particular instance:

```

User.observe('afterCreate',function(user){
  console.log('User with id of ' + user.id + ' was created.');
```

```

});

var u = User.find(5);
u.observe('afterDestroy',function(){
  //this particular user was destroyed
});

```

In the example above, each user that is created will be passed to the first callback. You can also call stopObserving() to remove a given observer, and use the observeOnce() method (same arguments as observe()) method if needed. Alternately, each event name is also a convenience method and the following example is functionally equivalent to the prior example:

```

User.afterCreate(function(user){
  console.log('User with id of ' + user.id + ' was created.');
```

```

});

var u = User.find(5);

```

```
u.afterDestroy(function(){
    //this particular user was destroyed
});
```

You can stop the creation, saving or destruction of a record by throwing the \$break variable inside any observers of the beforeCreate, beforeSave and beforeDestroy events respectively:

```
User.beforeDestroy(function(user){
    if(!allow_deletion_checkbox.checked){
        throw $break; //record will not be destroyed
    }
});
```

To observe a given event on all models, you can do the following:

```
ActiveRecord.observe('created',function(model_class,model_instance){});
```

afterFind works differently than all of the other events. It is only available to the model class, not the instances, and is called only when a result set is found. A find first, or find by id call will not trigger the event.

```
User.observe('afterFind',function(users,params){
    //params contains the params used to find the array of users
});
```

Validation

Validation is performed on each model instance when create() or save() is called. Validation can be applied either by using pre defined validations (validatesPresenceOf, validatesLengthOf, more will be implemented soon), or by defining a valid() method in the class definition. (or by both). If a record is not valid, save() will return false. create() will always return the record, but in either case you can call getErrors() on the record to determine if there are any errors present.

```
User = ActiveRecord.define('users',{
    username: '',
    password: ''
},{
    valid: function(){
        if(User.findByUsername(this.username)){
            this.addError('The username ' + this.username + ' is already taken.');
        }
    }
});

User.validatesPresenceOf('password');

var user = User.build({
    'username': 'Jessica'
});

user.save(); //false
var errors = user.getErrors(); //contains a list of the errors that occurred
user.set('password','rabbit');
```

```
user.save(); //true
```

Relationships

Relationships are declared with one of three class methods that are available to all models:

- belongsTo
- hasMany
- hasOne

The related model name can be specified in a number of ways, assuming that you have a Comment model already declared, any of the following would work:

```
User.hasMany(Comment)
User.hasMany('Comment')
User.hasMany('comment')
User.hasMany('comments')
```

Each relationship adds various instance methods to each instance of that model. This differs significantly from the Rails "magical array" style of handling relationship logic:

Rails:

```
u = User.find(5)
u.comments.length
u.comments.create :title => 'comment title'
```

ActiveRecord.js:

```
var u = User.find(5);
u.getCommentList().length;
u.createComment({title: 'comment title'});
```

Missing Features

ActiveRecord.js will not support all of the advanced features of the Ruby ActiveRecord implementation, but several key features are currently missing and will be added soon:

- complete set of default validations from ActiveRecord::Validations::ClassMethods
- ActsAsList
- ActsAsTree
- hasMany :through (which will likely be the only supported many to many relationship)

More examples and unit tests can be found at the **source repository**.

ActiveJS is a trademark of Aptana, Inc. ©2008 **Aptana Inc.**